

DAYANANDA SAGAR UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF ENGINEERING



DAYANANDA SAGAR UNIVERSITY
HAROHALLI, KANAKAPURA ROAD,
BENGALURU SOUTH Dt. - 562112

MINOR PROJECT

ON

INTELLIGENT ACCIDENT DETECTION AND ALERT SYSTEM USING MACHINE LEARNING

6th SEMESTER

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING

Submitted by

Arpita	(ENG23CS0024)
Bhuvana A M	(ENG23CS0037)
Darshini R	(ENG23CS0053)

Under the supervision of
Dr. Shreekant Salotagi
Assistant Professor,
Computer Science and Engineering

(2025-2026)



**DAYANANDA SAGAR
UNIVERSITY**

Department of Computer Science & Engineering

Harohalli, Kanakapura Road,
Bengaluru South Dt. – 562112
Karnataka, India

CERTIFICATE

This is to certify that the Mini-project work titled “**Intelligent Accident Detection and Alert System using Machine Learning**” is carried out by **Arpita (ENG23CS0024)**, **Bhuvana A M (ENG23CS0037)**, **Darshini R (ENG23CS0053)** Bonafide students of Bachelor of Technology in Computer Science and Engineering at the School of Engineering, Dayananda Sagar University, Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

Dr. Shreekant Salotagi

Assistant Professor Dept. of
CSE,

School of Engineering
Dayananda Sagar University

Dr. Girisha G S

Chairman CSE
School of Engineering

Dayananda Sagar
University

**Dr. Udaya Kumar
Reddy K R**

Dean
School of Engineering

Dayananda Sagar
University

Date:30-04-2026

DECLARATION

We ,**Arpita (ENG23CS0024), Bhuvana A M (ENG23CS0037), Darshini R (ENG23CS0053)** are students of sixth semester B. Tech in **Computer Science and Engineering**, at School of Engineering, **Dayananda Sagar University**, hereby declare that the Mini Project titled “**Intelligent Accident Detection and Alert System using Machine Learning**” has been carried out by us and submitted in partial fulfilment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**.

Students

Name : Arpita

USN: ENG23CS0024

Name :Bhuvana A M

USN: ENG23CS0037

Name : Darshini R

USN: ENG23CS0053

Signature

Place: Bengaluru

Date:30-04-2026

ACKNOWLEDGEMENT

It is a great pleasure for us to acknowledge the assistance and support of many individuals who have been responsible for the successful completion of this project work.

First, we take this opportunity to express our sincere gratitude to the School of Engineering & Technology, **Dayananda Sagar University Bengaluru** for providing us with a great opportunity to pursue our Bachelor's degree in this institution.

We would like to thank **Dr. Udaya Kumar Reddy K R**, Dean, School of Engineering & Technology, Dayananda Sagar University for his constant encouragement and expert advice.

It is a matter of immense pleasure to express our sincere thanks to **Dr. Girisha G S**, Department Chairman, Computer Science and Engineering, Dayananda Sagar University, for providing the right academic guidance that made our task possible.

We would like to thank our guide **Dr. Shreekant Salotagi** Assistant Professor, Dept. of Computer Science and Engineering, Dayananda Sagar University, for sparing his/her valuable time to extend help in every step of our project work, which paved the way for smooth progress and fruitful culmination of the project.

We are also grateful to our family and friends who provided us with every requirement throughout the course. We would like to thank one and all who directly or indirectly helped us in the Project work.

TABLE OF CONTENTS

	Pages
LIST OF ABBREVIATIONS.....	v
LIST OF FIGURES.....	vi
ABSTRACT.....	1
CHAPTER 1 INTRODUCTION	2
1.1.OBJECTIVE	3
1.2.SCOPE.....	3
CHAPTER 2 PROBLEM DEFINITION	4
CHAPTER 3 LITERATURE SURVEY.....	5-6
CHAPTER 4 REQUIREMENTS.....	7
4.1. FUNCTIONAL REQUIREMENTS.....	7
4.2.NON-FUNCTIONAL REQUIREMENTS	8
4.3.HARDWARE AND SOFTWARE REQUIREMENTS.....	8
CHAPTER 5 DESIGN AND METHODOLOGY	9-11
CHAPTER 6 RESULTS AND DISCUSSION	12-18
CHAPTER 7 CONCLUSION AND FUTURE WORK	19
7.1. CONCLUSION.....	19
7.2. SCOPE FOR FUTURE WORK.....	20
REFERENCES	21
SAMPLE CODE.....	22-24

LIST OF ABBREVIATIONS

Abbreviation	Full-form
ML	Machine Learning
CNN	Convolutional Neural Network
DL	Deep Learning
CV	Computer Vision
API	Application Programming Interface
GPS	Global Positioning System

LIST OF FIGURES

Fig. No.	Description	Pg. No.
4.1	Design flow of accident detection and alert system	9
6.1	Training vs Validation Loss Graph	12
6.2	Predicted Probability vs Actual Value	13
6.3	Distribution of Prediction Across Models	14
6.4	Model Accuracy Comparison Graph	14
6.5	Normalized Comparison of ML Models	16
6.6	Accident Detection Result with Location-Based Alert Message	18

ABSTRACT

Road accidents are a major cause of injury and death worldwide, often worsened by delays in emergency response. This project presents an intelligent accident detection and alert system that uses deep learning techniques to identify road accidents in real time from traffic surveillance camera feeds. A convolutional neural network (CNN)-based model, trained on labeled images of accident and non-accident scenarios, is employed to automatically classify visual data. Multiple architectures such as CNN, MobileNetV2, ResNet50, VGG16, and EfficientNetB0 were trained and evaluated, with the best-performing model selected for deployment.

The proposed system is integrated with traffic monitoring cameras, where video frames are continuously analyzed by the trained model. Upon detecting an accident, the system retrieves the geographical coordinates of the corresponding camera location and converts them into a human-readable address using geolocation services. An automated alert is then generated and sent via a communication platform, including essential details such as exact coordinates, address, and a navigation link to the accident site. This enables faster response from nearby hospitals and emergency services.

The system aims to reduce the time taken to report accidents and initiate rescue operations, thereby potentially saving lives. With further enhancements such as real-time video processing, dynamic hospital selection, and integration with smart city infrastructure, this solution can be scaled for practical deployment in urban traffic management systems. The proposed system is designed to be scalable and adaptable for smart city environments, where multiple surveillance cameras can be monitored simultaneously through a centralized control system. By leveraging lightweight and efficient deep learning models, the solution can be deployed on edge devices or cloud platforms for real-time performance. Furthermore, the system can be extended to include features such as accident severity analysis, automatic ambulance dispatch, and integration with traffic signal control to create a green corridor for emergency vehicles. This enhances the overall effectiveness of urban traffic management and contributes to building safer and more responsive transportation systems:

CHAPTER 1

INTRODUCTION

Road accidents have become a serious global issue, resulting in a large number of injuries and fatalities every year. One of the major challenges in accident management is the delay in detecting accidents and providing immediate medical assistance. In many cases, the lack of timely response increases the severity of injuries and reduces the chances of survival. Therefore, there is a need for an intelligent system that can automatically detect accidents and quickly alert emergency services.

With the rapid development of technologies in Machine Learning and Computer Vision, it has become possible to analyze visual data and make accurate predictions in real time. These technologies enable systems to process images and videos efficiently, making them suitable for applications such as accident detection, surveillance, and safety monitoring.

This project presents an accident detection and alerting system that uses deep learning techniques to identify accident scenarios from video input. The system captures real-time frames from a camera, processes them, and classifies them as accident or non-accident using a trained model. This automated approach reduces the dependency on human observation and improves detection accuracy.

Once an accident is detected, the system immediately retrieves the location details using geolocation services and sends an alert message to nearby hospitals or emergency contacts. The alert includes important information such as coordinates, address, and navigation links, which helps responders reach the accident spot quickly. This significantly reduces response time and improves the chances of saving lives.

The proposed system aims to enhance road safety by integrating intelligent detection with real-time alert mechanisms. By combining data processing, predictive modeling, and communication technologies, the system provides an efficient and reliable solution for accident management. It demonstrates how modern technologies can be used effectively to address real-world problems and improve public safety.

1.1 OBJECTIVE

- To develop an automated accident detection system using deep learning techniques.
- To analyze real-time video input from traffic surveillance cameras.
- To accurately classify scenes into accident and non-accident categories.
- To reduce the delay in accident reporting by detecting incidents instantly.
- To identify the exact location of the accident using camera coordinates.
- To convert geographical coordinates into a readable address using geolocation services.
- To send immediate alerts to emergency services such as hospitals and ambulances.
- To integrate the system with communication platforms like WhatsApp for real-time notification.
- To minimize human intervention in monitoring traffic accidents.
- To improve road safety by enabling faster emergency response.

1.2 SCOPE

- Implementation of an AI-based accident detection system using CCTV/traffic cameras.
- Real-time monitoring and analysis of video streams for accident detection.
- Use of trained deep learning models (CNN, MobileNetV2, VGG16, etc.) for classification.
- Integration of location mapping to determine exact accident position.
- Automatic alert generation with coordinates, address, and Google Maps link.
- Application in smart city infrastructure and highway monitoring systems.
- Scalability to handle multiple cameras and centralized monitoring.
- Future integration with nearest hospital detection and ambulance dispatch systems.
- Extension to detect other road events like traffic congestion and violations.
- Potential deployment on edge devices or cloud platforms for real-time performance.

CHAPTER 2

PROBLEM DEFINITION

Road traffic accidents have become a major public safety concern, leading to a significant number of injuries and fatalities every year. One of the most critical challenges in accident management is the delay in detecting the incident and informing emergency services. In many situations, accidents occur in areas where immediate help is not available, and victims are unable to report the incident due to injuries or lack of communication facilities. This delay in response time often results in severe consequences, including loss of life that could have been prevented with timely medical assistance.

Traditional accident detection methods rely heavily on human intervention, eyewitness reports, or manual monitoring of surveillance systems. These approaches are inefficient, time-consuming, and prone to errors, as continuous human observation is not always feasible. Additionally, sensor-based systems installed in vehicles may fail due to hardware limitations or may not be present in all vehicles, making them less reliable for widespread implementation.

Although recent advancements in technology have introduced intelligent systems for monitoring traffic, many existing solutions either focus on accident prevention or lack real-time detection and alert mechanisms. Systems based on driver behavior or environmental conditions may not accurately detect accidents after they occur. Similarly, computer vision-based systems require robust models and efficient processing to handle real-time video data effectively.

Therefore, there is a need for an automated, reliable, and real-time accident detection system that can analyze traffic surveillance data, accurately identify accident scenarios, and immediately notify emergency services with precise location details. Such a system can significantly reduce response time, improve emergency coordination, and ultimately save lives. The proposed project aims to address these challenges by developing a deep learning-based accident detection and alert system integrated with location and communication technologies.

CHAPTER 2

LITERATURE SURVEY

In the above-discussed system, its working principle is hardware-based. If the user takes more than he

should take, the system automatically stops the engine when the vibrations get recorded. When there are variations in the analyzed data, a message is transmitted that ensures safety for the driver by remaining safe for about thirty seconds. The driver must press a button after this, otherwise, an alert message would be transmitted to the authorities.[1]

The purpose of this system is to observe continuously where the face of the driver will be monitored when the car starts and the blinkings of eyes are being monitored in order to detect whether the driver falls asleep or not. If the system detects any change in the analyzed data, it will send contact information to hospitals, police station and other emergency services [2].

The design of a system that will be able to detect accidents in real time by analyzing live CCTV camera footage. This device will analyze the environment and once it identifies anything that looks questionable, the system will then determine if an accident has taken place using imagery [3].

Design of a system that will take inputs from the driver in terms of weather, nature of the road, vehicle speed, and lighting conditions, after which the device will inform the driver if the road conditions are favorable or not, in addition to predicting possible accidents [4].

The system uses sensors which will continuously monitor the environmental data and relay it to the microcontroller which then analyzes the data and determines if there is an accident occurring. Once this is determined, the local alarm will be set off, the data will be sent to the Raspberry Pi which will send out the data through WiFi to the cloud for analysis by the machine learning model. If any emergencies arise, the device will alert the driver and send out the data to rescue teams via cell phone [5]

The system works by initially feeding it a video which it breaks down into frames that are then checked for the presence of features that can help in object recognition. The frames are subjected to a detection algorithm like YOLO which is used to analyze the video to find out if there was an accident. This procedure has already been tested through training and testing on annotated data. Once the accident is detected, the program notifies the administrator of the event together with pictures and the

number of injuries alongside storing the data in a database [6].

In this case, both the video frame and optical flow become the inputs to the ConvLSTM layers that help in extracting the spatial and temporal characteristics. After convolution of the inputs, there is a batch normalization of the data, and GAP is done in order to make the inputs smaller. Then the generated outputs are further made smaller in order to get the RGB and motion features, which will be helpful in determining what kind of activity is going on in the video[7].

All this is done based on the principles of operation of the car tracking system and warning system. First, over the entire system. Data acquisition takes place through the speed sensor, while the GPS module will provide location coordinates through UART communication. The obtained GPS data are processed by the microcontroller and then shown on the LCD display. Other necessary data are stored in the EEPROM memory. On the other hand, the GSM module is used to send data to an external destination or for sending SMS[8].

It too is akin to the IoT based surveillance system which uses MQTT protocol for transmitting the data. Firstly, data about motion and distances is captured by means of an accelerometer and ultrasonic sensor. After that, this data is sent to the ESP8266 Wi-Fi module. The ESP8266 will then send this data obtained from the sensor via LOSANT. There isn't much that can be done with this data except for the purpose of triggering an alarm, creating a data visualization dashboard, executing many other processes including sending emails, and even subscribing ESP8266 to a certain topic and sending messages to it[9].

To perform any operation in the system, the video is divided into shots with the help of PySceneDetect. Further, the shots are divided into frames with the help of OpenCV. In this way, only useful information is extracted. Further, the system performs feature extraction from the key frames with the help of ResNet50 architecture. These features are then classified with the help of K-Means Clustering, creating a "bag of visual words". Finally, the data is divided into training and testing data, where SVM classification is performed for accident detection in the video [10].

The latest innovations in the field of deep learning include the improvement of image classification problems through the use of CNNs. It allows for automatic feature detection from images and makes it unnecessary for engineers to create handcrafted features in order for them to outperform other traditional machine learning methods. One other innovation the area includes transfer learning using VGG16, ResNet50, and MobileNetV2 which already learned from large datasets like ImageNet [11].

CHAPTER 4

REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

- Capture live video input from traffic surveillance cameras.
- Process video frames and convert them into image data for analysis.
- Load and use the trained deep learning model (.h5) for prediction.
- Detect accident and non-accident scenarios using the ML model.
- Identify the camera location where the accident occurred.
- Convert latitude and longitude into a readable address using geolocation.
- Generate accident alert message with coordinates, address, and map link.
- Send real-time alerts via WhatsApp using Twilio API.
- Display system output and alert status on the console or interface.
- Handle errors such as missing location or failed message delivery.

4.2 NON-FUNCTIONAL REQUIREMENTS

- The system should provide real-time or near real-time accident detection.
- The system must be reliable and provide accurate predictions.
- It should be scalable to support multiple cameras simultaneously.
- The system should ensure fast response time for alert generation.
- User data and API credentials must be securely handled.
- The system should be easy to maintain and update.
- It should have a user-friendly interface or simple execution flow.
- The system must be efficient in terms of memory and processing usage.
- It should work under different environmental conditions (day/night).
- The system should be compatible with standard hardware and software platforms.

4.3 HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Computer/Laptop (Minimum 8GB RAM recommended)
- Processor: Intel i5 or higher
- Webcam / CCTV Camera (for input)
- Internet connection (for API services and alerts)

Software Requirements:

- Python (programming language)
- TensorFlow / Keras (deep learning framework)
- OpenCV (image and video processing)
- NumPy (numerical computations)
- Pandas (data handling)
- Matplotlib (visualization)
- Geopy (location conversion)
- Geocoder (IP-based location fallback)
- Twilio API (WhatsApp alert system)
- Jupyter Notebook / Google Colab (model training)
- VS Code (development environment)

CHAPTER 4

DESIGN AND METHODOLOGY

DESIGN

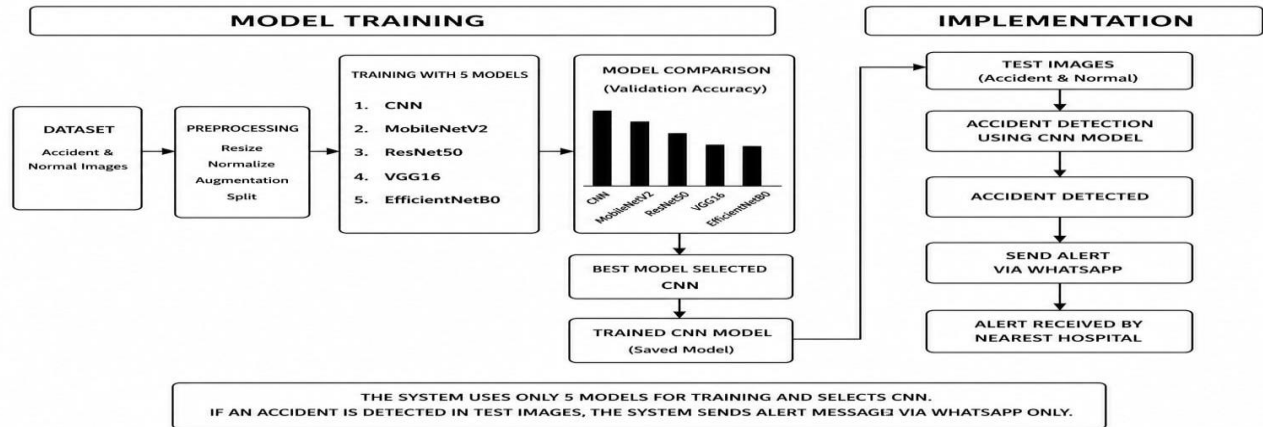


Fig4.1. Design flow of accident detection and alert system

The system is designed in two main phases: model training and implementation. In the training phase, a dataset of accident and normal images is collected and preprocessed using steps such as resizing, normalization, augmentation, and splitting into training and validation sets. This ensures better learning and generalization. Five deep learning models—CNN, MobileNetV2, ResNet50, VGG16, and EfficientNetB0—are trained on the same dataset. After training, the models are evaluated and compared based on validation accuracy, and the CNN model is selected as the best-performing model.

In the implementation phase, the trained CNN model is used to analyze test images. The system takes images as input and processes them through the model to detect accidents. The model extracts features from each image and determines whether an accident is present. This approach avoids live video processing and instead focuses on image-based detection, making the system simpler and efficient while still demonstrating accurate accident detection capability.

When an accident is detected, the system automatically triggers an alert. The alert message is sent via WhatsApp to the nearest hospital to ensure quick response and immediate attention. This automated alerting mechanism helps reduce delays in emergency situations and improves the chances of timely medical assistance. Overall, the system provides a clear workflow from data preparation and model selection to detection and real-time alert communication.

METHODOLOGY

1. Data Collection

Data collection is the initial step in this project. The dataset consists of images representing accident and non-accident scenarios collected from various online sources. These images reflect real-world road conditions, including vehicle collisions, traffic situations, and normal driving scenes. The dataset is used to train and evaluate the deep learning model for accurate accident detection.

2. Data Preprocessing

Data preprocessing is performed to prepare the dataset for model training. This includes resizing all images to a fixed dimension (224×224), normalizing pixel values, and organizing the data into training and validation sets. Data augmentation techniques such as rotation, flipping, and scaling are applied to improve model generalization and reduce overfitting. This step ensures better performance and robustness of the model.

3. Feature Extraction

Feature extraction is carried out using deep learning models that automatically learn important visual patterns from the images. Instead of manually selecting features, convolutional layers extract relevant information such as edges, shapes, and object structures. Pre-trained models like MobileNetV2, ResNet50, and VGG16 are used to enhance feature extraction efficiency and accuracy.

4. Model Training

In this phase, multiple machine learning models are trained using the prepared dataset. Models such as CNN, MobileNetV2, ResNet50, VGG16, and EfficientNetB0 are implemented and evaluated. Each model learns to classify images into accident or non-accident categories. Training is performed using optimization techniques like Adam optimizer and binary cross-entropy loss function.

5. Model Evaluation

The trained models are evaluated based on performance metrics such as accuracy, precision, recall, and F1-score. Validation data is used to test the model's ability to generalize to unseen data. Based

on the evaluation results, the best-performing model is selected for deployment in the system.

6. Accident Detection System

The selected model is integrated into the accident detection system. The system processes input from traffic cameras by converting video streams into frames. Each frame is passed through the trained model to predict whether an accident has occurred. If the prediction exceeds a predefined threshold, the system identifies it as an accident.

7. Location Identification

Once an accident is detected, the system retrieves the corresponding camera's geographical coordinates from a predefined database. These coordinates are then converted into a human-readable address using geolocation services such as Geopy. This helps in identifying the exact accident location.

8. Alert Generation

After identifying the accident location, the system generates an alert message containing important details such as coordinates, address, nearest hospital and a Google Maps link. This message is formatted clearly to ensure easy understanding by emergency responders.

9. Notification System

The alert message is sent to the nearest Hospital using the Twilio API via WhatsApp. This ensures real-time communication and faster response from hospitals, ambulances, or rescue teams. The system confirms successful message delivery or handles errors if any occur.

10. System Output and Monitoring

The system displays the detection result and alert status through the console or interface. It continuously monitors incoming data and processes it in real time. This ensures that any accident occurring within the camera's view is detected and reported immediately.

CHAPTER 6

RESULTS AND DISCUSSION

RESULTS

A. Training vs Validation Loss Graph

The above graph illustrates the variation of training loss and validation loss over multiple epochs during the training process of the model. It can be clearly observed that both the training loss and validation loss decrease steadily as the number of epochs increases, which indicates that the model is learning effectively from the dataset. The reduction in training loss shows that the model is improving its ability to fit the training data, while the simultaneous decrease in validation loss suggests that the model is also performing well on unseen data. The close proximity of the training and validation curves indicates that the model is not significantly overfitting, which is a desirable characteristic in machine learning models. Additionally, the smooth downward trend of both curves reflects stable learning and proper convergence of the model. This behavior confirms that the chosen architecture and training process are suitable for the given dataset and that the model has achieved a good balance between bias and variance.



Fig6.1. Training vs Validation Loss Graph

B. Predicted Probability vs Actual Value

The above graph represents the relationship between the predicted probabilities generated by the model and the actual class labels of the dataset. The predicted probabilities are plotted as scattered points, while the actual labels are used as a reference for comparison. It can be observed that a majority of the predicted values are closely aligned with their corresponding actual labels, indicating that the model is capable of accurately distinguishing between accident and non-accident scenarios. The clustering of predicted values near the correct class boundaries suggests that the model has learned meaningful patterns from the data. However, a few scattered points that deviate from the actual labels can be observed, which may be due to ambiguous images, noise in the dataset, or complex real-world conditions. Despite these minor deviations, the overall distribution of predictions demonstrates that the model has strong classification capability and is reliable for practical implementation. This graph validates the effectiveness of the trained model in predicting outcomes with high confidence.

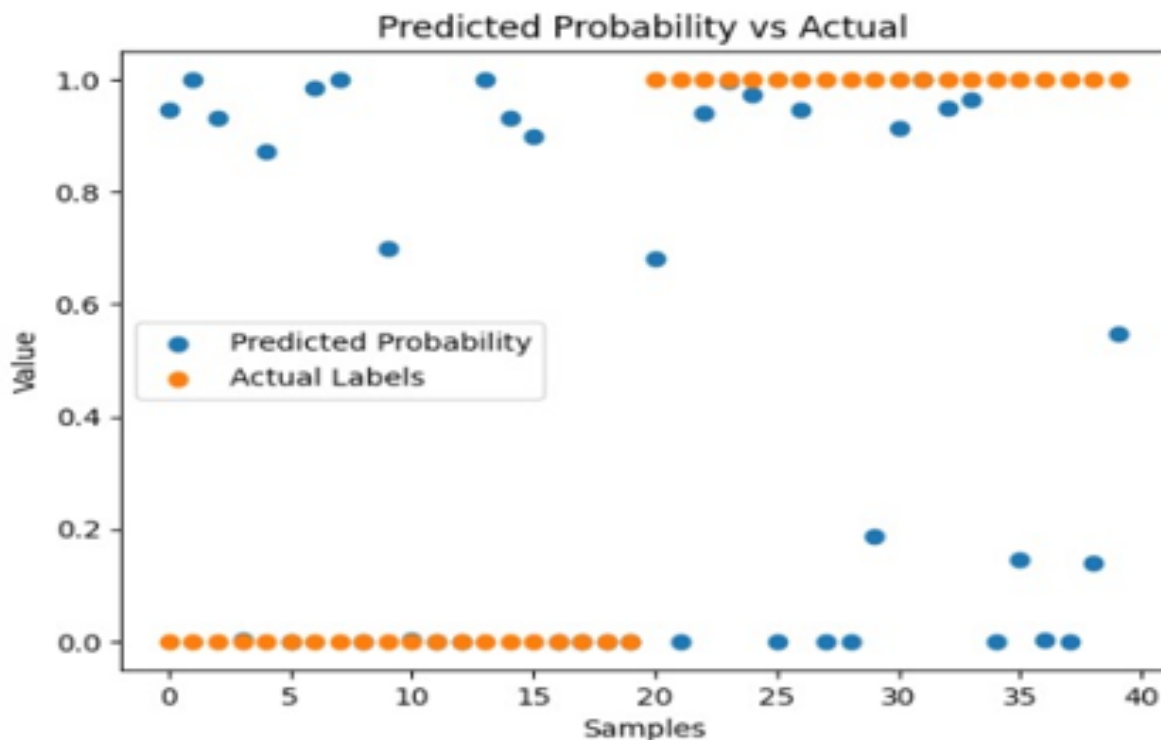


Fig.6.2. Predicted Probability vs Actual Value

C. Distribution of Prediction Across Models

The above figure presents the distribution of prediction probabilities across different machine learning models used in the project. It provides a comparative visualization of how each model behaves in terms of prediction confidence and consistency. The spread of values in the box plots indicates the variability in predictions made by each model. Models such as MobileNetV2 and VGG16 show tighter distributions with higher median values, suggesting that they produce more consistent and confident predictions. In contrast, models like CNN and EfficientNetB0 exhibit a wider spread, indicating less stability and lower confidence in predictions. This variation highlights the differences in feature extraction capabilities and learning efficiency among the models. The figure clearly demonstrates that pre-trained models tend to perform better due to their ability to capture complex visual patterns. Overall, this distribution analysis helps in understanding the reliability and robustness of each model in accident detection tasks.

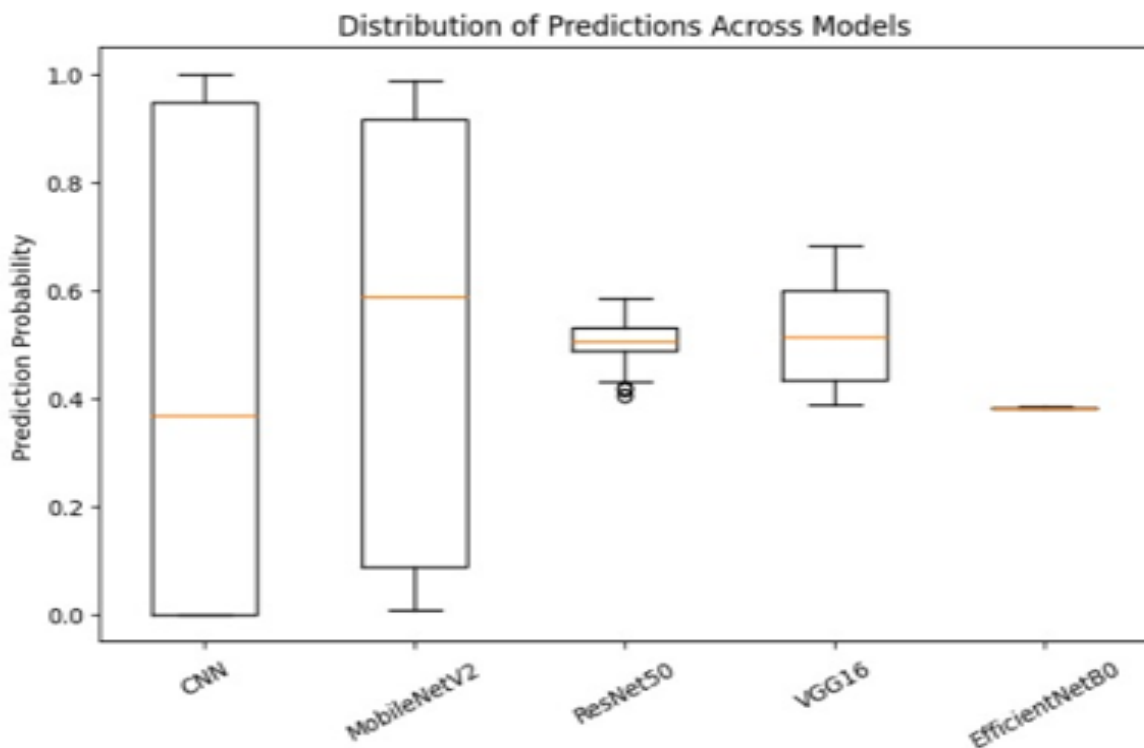


Fig.6.3. Distribution of Prediction Across Models

D. Normalized Comparison of ML Models

The above graph provides a normalized comparison of different machine learning models based on key evaluation metrics such as accuracy, precision, recall, and F1-score. Normalization allows all metrics to be represented on a common scale, making it easier to compare the overall performance of each model. It can be observed that models like MobileNetV2 and VGG16 achieve consistently high values across all metrics, indicating strong performance in correctly identifying both accident and non-accident cases. These models demonstrate a good balance between precision and recall, which is essential for minimizing false positives and false negatives. On the other hand, models such as CNN and EfficientNetB0 show comparatively lower values, indicating weaker performance in certain metrics. This comparison clearly highlights the superiority of pre-trained models over basic architectures. The graph plays a crucial role in selecting the most suitable model for deployment by considering multiple performance factors rather than relying on a single metric.

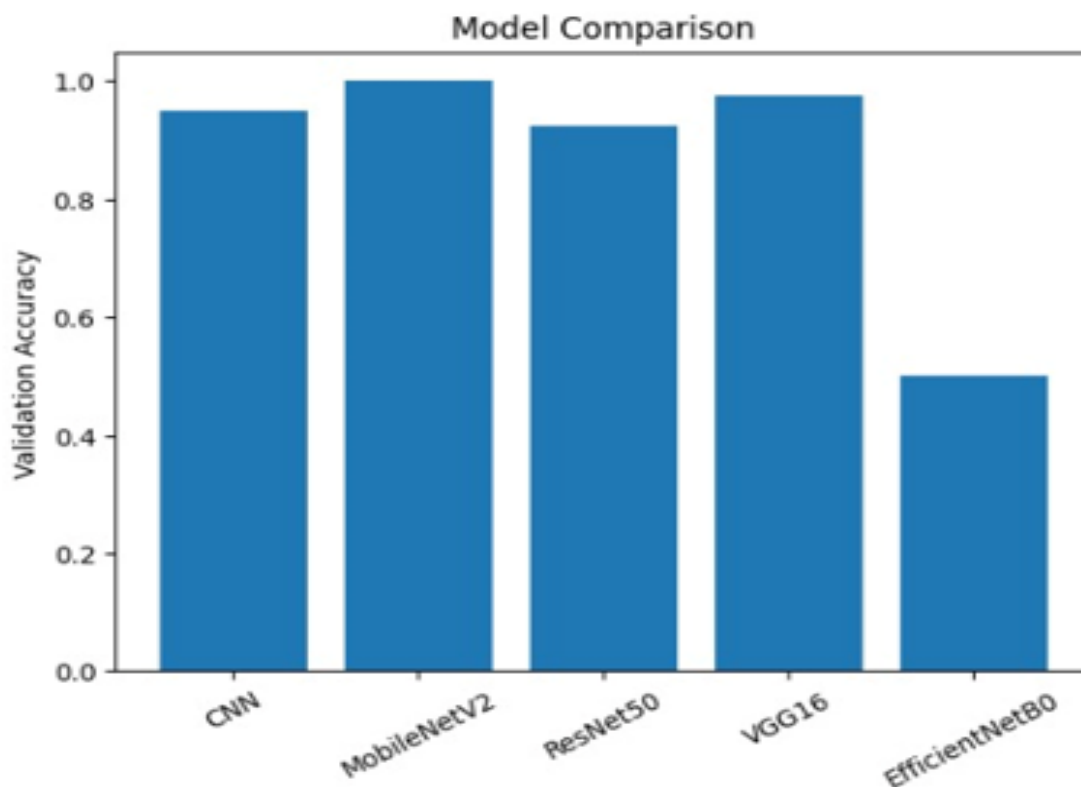


Fig6.4. Model Accuracy Comparison Graph

E. Model Accuracy Comparison Graph

The above bar graph illustrates the comparison of validation accuracy achieved by different machine learning models used in this project. It is evident from the graph that VGG16 achieves the highest accuracy among all the models, followed closely by MobileNetV2 and ResNet50. This indicates that pre-trained deep learning models are more effective in extracting relevant features and making accurate predictions. In contrast, the basic CNN model and EfficientNetB0 show relatively lower accuracy, which may be due to limited training data or insufficient feature extraction capability. The graph clearly demonstrates the advantage of using transfer learning techniques, where pre-trained models leverage knowledge from large datasets to improve performance on specific tasks. Based on this comparison, VGG16 is selected as the best-performing model for the accident detection system. This result supports the effectiveness of deep learning approaches in solving real-world problems related to road safety and accident detection.

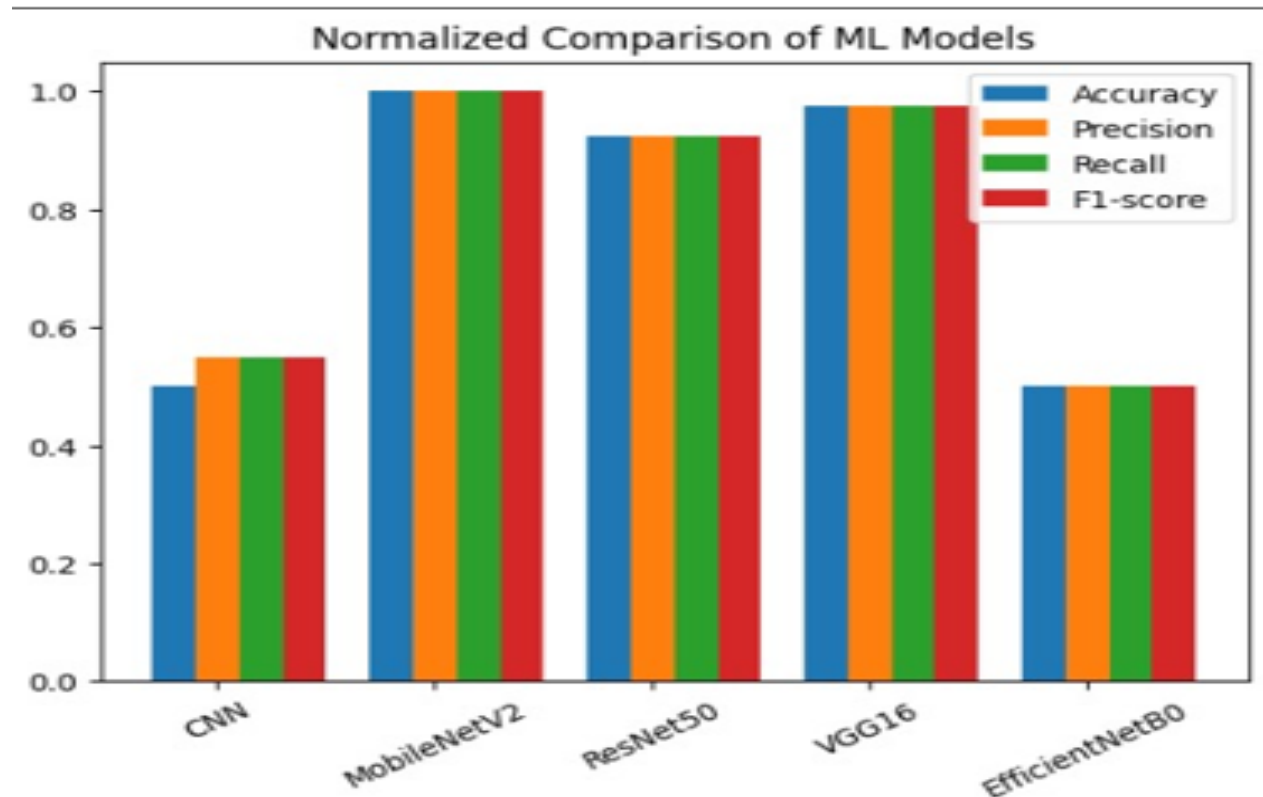


Fig.6.5. Normalized Comparison of ML Models

F. Accident Detection Result with Location-Based Alert Message

The above figure illustrates the final output of the proposed accident detection and alert system, demonstrating its ability to function as a complete solution. When an accident is detected by the trained deep learning model, the system automatically triggers the alert mechanism without any human intervention. It retrieves the exact geographical coordinates of the accident location and converts them into a readable address using geolocation services. The system then generates an alert message containing essential details such as coordinates, address, nearest hospital, and a Google Maps navigation link. This alert is sent via WhatsApp using the Twilio API, ensuring real-time communication with emergency services. The successful delivery of the message confirms that the system can provide immediate and accurate information to responders. This helps reduce response time and enables quicker assistance at the accident site. Overall, the result validates the effectiveness and practical applicability of the proposed system.

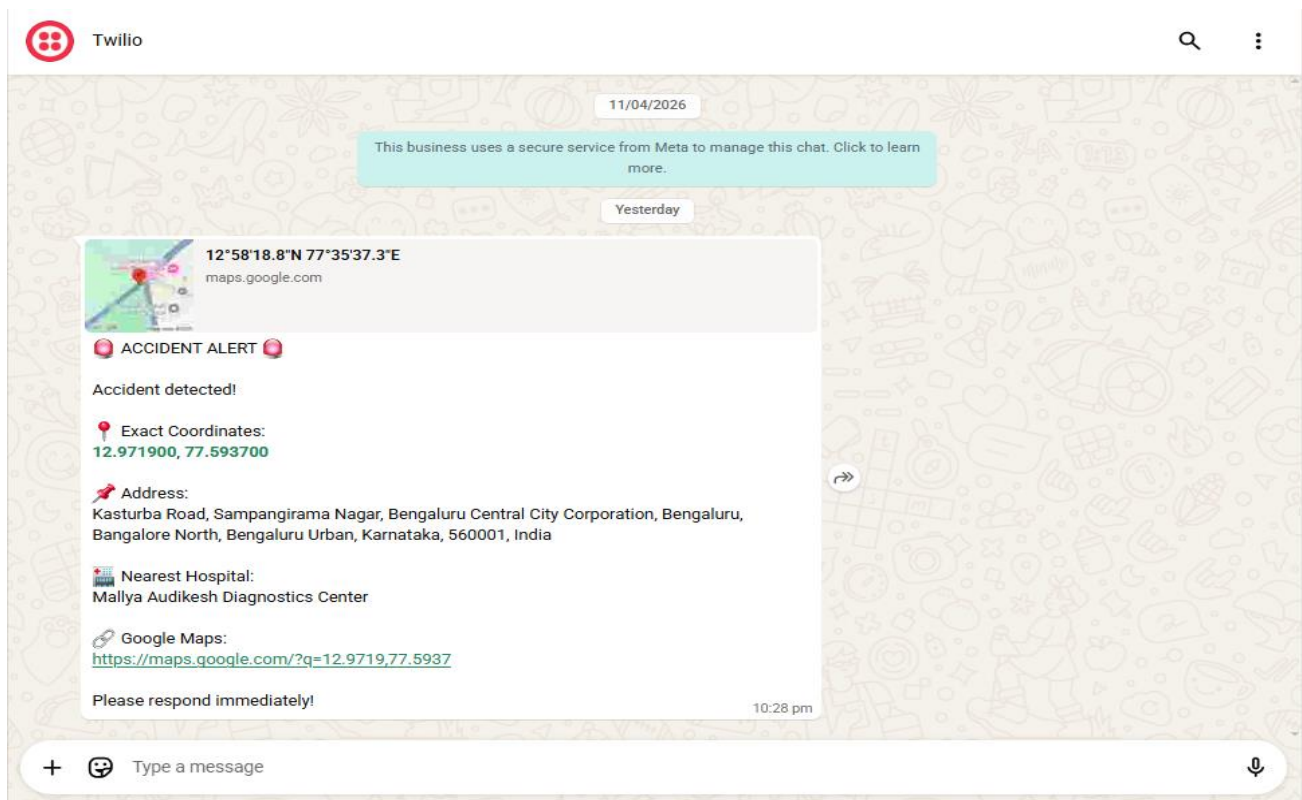


Fig6.6. Accident Detection Result with Location-Based Alert Message

DISCUSSION

The results obtained from the proposed accident detection system demonstrate that deep learning techniques are effective in identifying road accidents from visual data. The trained models were able to distinguish between accident and non-accident scenarios with good accuracy, indicating that feature extraction using convolutional layers is reliable for this type of problem. Among the models evaluated, pre-trained architectures showed better performance compared to basic models due to their ability to capture complex visual patterns.

The analysis of model performance suggests that the system learns effectively from the dataset; however, slight differences between training and validation results indicate the possibility of overfitting. This can be attributed to the limited size of the dataset used for training. Increasing the dataset size and incorporating more diverse images, such as different weather conditions and lighting variations, can improve the model's generalization capability and make it more robust for real-world scenarios.

The system also demonstrated the ability to correctly classify most test images, showing its potential for practical implementation. However, certain limitations exist when dealing with low-quality images, occlusions, or heavy traffic conditions, which may affect prediction accuracy. These challenges highlight the importance of using high-quality and diverse datasets for training.

The integration of the alert system using Twilio was successful, as the system generated and delivered real-time WhatsApp notifications containing accident details such as coordinates, address, and navigation link. This confirms that the proposed system is capable of providing immediate communication to emergency services, thereby reducing response time. Overall, the system performs effectively as a prototype and shows strong potential for real-world deployment with further improvements in scalability, dataset size, and real-time processing capabilities.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The proposed accident detection and alert system demonstrates the effective use of deep learning techniques for identifying road accidents from traffic surveillance data. The system processes video input, converts it into frames, and uses trained models to classify accident and non-accident scenarios. Among the models tested, pre-trained architectures showed better performance due to their ability to extract complex visual features. This confirms that deep learning is a reliable approach for automated accident detection.

In addition to detection, the system integrates location mapping and communication modules to provide a complete solution. Once an accident is detected, the system retrieves the camera coordinates, converts them into a readable address, and generates an alert message. The integration of Twilio API enables real-time WhatsApp notifications containing location details and navigation links. This ensures that emergency services can respond quickly and efficiently, reducing delays in providing assistance.

Overall, the system performs well as a prototype and successfully achieves its objective of automating accident detection and alert generation. Although certain limitations exist, such as dataset size and real-time processing challenges, the system provides a strong foundation for future improvements. With further enhancements, it can be developed into a scalable solution for smart traffic management and improved road safety.

7.2 SCOPE FOR FUTURE WORK

The proposed system can be significantly improved by enhancing the dataset used for training the deep learning models. Currently, the performance of the system depends on the quality and diversity of the dataset, which may be limited in size and variations. In future work, a larger and more diverse dataset can be collected, including images from different weather conditions such as rain, fog, and night-time scenarios, as well as various traffic densities and camera angles. This will help the model generalize better and improve its accuracy in real-world situations. Additionally, advanced data augmentation techniques can be applied to further strengthen the robustness of the model.

Another important area for improvement is the implementation of real-time video processing and system optimization. The current system can be enhanced by integrating efficient frameworks such as OpenCV for continuous video streaming and processing. Lightweight models like MobileNet can be optimized for faster predictions, making the system suitable for deployment on edge devices such as Raspberry Pi or embedded systems. Furthermore, the system can be extended to support multiple cameras simultaneously, enabling centralized monitoring of large areas such as cities and highways. This would make the solution scalable and more practical for real-world applications.

The system can also be expanded by integrating additional intelligent features to improve emergency response. Instead of using a predefined hospital, future versions can include dynamic nearest hospital detection using Google Maps API. The system can also be connected with traffic signal control to automatically create a green corridor for ambulances. Other enhancements may include accident severity detection, automatic emergency calling, and integration with police and rescue services. With cloud integration and IoT support, the system can evolve into a complete smart city solution for efficient traffic monitoring and rapid emergency response.

REFERENCES

1. G.Siri,G.Priyanka,B.Divya,G.Sameera,M.Pallavi,“Accident Detection and Alert System” IRJETS vol.04,Issue06,June-2022.
2. Sinchana.RD,Sindhu.KS,Sudhakara.HM, “Accident Detection and Alert System for enhanced Safety and Response”, vol.13,Issue11,2024.
3. Priyadarshini.C.patil,Padmanabhan Ghanate, Rishikesh patil, “Road Accident Detection and Alert Notification System using Machine Learning ”, IJSDR vol.10,Issue12, 2025.
4. Devisha Arunadevi Tiwari,Emmadi Bhanu Sai,Bandapu Sai Chandu,”Machine Learning based Accident detection and Alert System” JEITR, May 2024.
5. Gopal Sunil Rathod,Rani Chandrabhan Tipale,Dr. Kapil Jajulwar,”Intelligent Accident Detection and Alerting System based on Machine Learning over the IOT Network”2022 International Conference on Futuristic Technologies (INCOFT) Karnataka, India. Nov 25-27, IEEE 2022.
6. Swapnali.R.Pawar,Rushikesh Bamne,Shreyash Patil,Sushant Chipate,Tanmay Vasagadekar,”Detection and Alert generation of road accident using Deep Learning”International Conference on Healthcare Innovations,Software and Enginnering Technologies(HISET),IEEE 2024.
7. VICTOR A.ADEWOPO,NELLY ELSAYED,”Smart City Transporation: Deep Learning Ensemble Approach for Traffic Accident Detection “,IEEE 2024.
8. Pritam Banik Roy,Draksha,Ankika Dey,Rajendrani Mukherjee,Suparna Biswas,Suvamoy Changder,”Vehicle Tracking and Accident Prevention using Smart Technologies”,21st India Council International Conference (INDICON),IEEE 2024.
9. Ravi K Kodali,Shubhi Sahu,”MQTT Based Vehicle Accident Detection and Alert System”,IEEE 2017.
10. Aman Kumar Agrawal, Kadamb Agarwal,Jitendra Choudhary, Aradhita Srihita Tangudu,Nishkarsh Makhija, Rajitha B, “ Automatic and SVM”,Fifth International Conference on Research in Computational. Intelligence and Communication Networks(ICRCICN),IEEE 2020.
11. Arnav Chaudhari, Harsh Agrawal, Smiti Poddar, Kiran Talele, Manisha Bansode "Smart Accident Detection And Alert System" IEEE India Council International Subsection Conference(INDISCON) 2021.

SAMPLE CODE

DETECT.PY CODE

```
import cv2
import numpy as np
from tensorflow.keras.models import load_model
from alert import send_alert
cap = cv2.VideoCapture(0)
model = load_model("accident_model.keras")
# Prevent repeated alerts
alert_sent = False
ACCIDENT_LAT = 12.9719
ACCIDENT_LNG = 77.5937
while True:
    ret, frame = cap.read()
    if not ret:
        break
    img = cv2.resize(frame, (224, 224))
    img = img / 255.0
    img = np.reshape(img, (1, 224, 224, 3))
    pred = model.predict(img)
    if pred < 0.5:
        label = "ACCIDENT"
        if not alert_sent:
            print("Accident Detected! Sending Alert...")
            send_alert(ACCIDENT_LAT, ACCIDENT_LNG)
            alert_sent = True
        color = (0, 0, 255)
    # Red
    else:
        label = "NO ACCIDENT"
        alert_sent = False
        color = (0, 255, 0) # Green
    cv2.putText(
        frame,
        label,
        (50, 50),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        color,
        2
    )
    cv2.imshow("Detection", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

```
cap.release()
cv2.destroyAllWindows()
```

ALERT.PY CODE

```
import geocoder
from twilio.rest import Client
from geopy.geocoders import Nominatim
account_sid = "YOUR_ACCOUNT_SID"
auth_token = "YOUR_AUTH_TOKEN"
client = Client(account_sid, auth_token)
def get_address(lat, lng):

    try:
        geolocator = Nominatim(user_agent="accident_app")
        location = geolocator.reverse((lat, lng), language='en')
        return location.address if location else "Address not found"

    except:
        return "Address not available"
def send_alert(lat=None, lng=None):
    if lat is None or lng is None:
        g = geocoder.ip('me')
        lat, lng = g.latlng
    location_text = f'{lat:.6f}, {lng:.6f}'
    address = get_address(lat, lng)
    hospital = "Mallya Audikesh Diagnostics Center"
    link = f"https://maps.google.com/?q={lat},{lng}"
    message_body = f"""
```

ACCIDENT ALERT

```
Accident detected!
Exact Coordinates:
{location_text}
Address:
{address}
Nearest Hospital:
{hospital}
Google Maps:
{link}
Please respond immediately!
"""
```

```
try:
    message = client.messages.create(
        from_='whatsapp:+14155238886',
        body=message_body,
```

```
        to='whatsapp:+91XXXXXXXXXX'
    )
    print("Alert Sent Successfully!")
    print("Message SID:", message.sid)

except Exception as e:
    print("Failed to send alert:", e)

if __name__ == "__main__":
    accident_lat = 12.9719
    accident_lng = 77.5937
    send_alert(accident_lat, accident_lng)
```

LOCATIONS.PY

```
CAMERA_LOCATIONS = {
    "cam_1": (12.9719, 77.5937),
```

